

Fome no Minhocao -Codigo Explicado

Nome completo: Arthur Carvalho Fontinele

Matricula: 252022430

Email: 252022430@aluno.unb.br

Este documento explica, parte por parte, a solucao do problema "Fome no Minhocao". O mapa (grid N x M) e representado como um grafo explicito: cada celula livre e um vertice, e existe uma aresta entre duas celulas vizinhas (cima, baixo, esquerda, direita) sempre que nenhuma delas for obstaculo. O grafo e construido como uma lista de adjacencia (um vetor de listas ligadas) e depois percorrido com busca em largura (BFS) para achar o caminho mais curto entre o aluno (A) e o RU (R).

Codigo completo

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_VERTICES 1000000

typedef struct Aresta {
    int destino;
    char movimento;
    struct Aresta *proxima;
} Aresta;

Aresta *listaDeAdjacencia[MAX_VERTICES];
char mapa[1000][1001];
int distancia[MAX_VERTICES];
int veioDe[MAX_VERTICES];
char movimentoParaChegar[MAX_VERTICES];
int fila[MAX_VERTICES];

int altura, largura;

int deltaLinha[4] = {-1, 1, 0, 0};
int deltaColuna[4] = {0, 0, -1, 1};
char letraDoMovimento[4] = {'C', 'B', 'E', 'D'};

void adicionarAresta(int origem, int destino, char movimento) {
    Aresta *nova = (Aresta *) malloc(sizeof(Aresta));
    nova->destino = destino;
    nova->movimento = movimento;
    nova->proxima = listaDeAdjacencia[origem];
    listaDeAdjacencia[origem] = nova;
}

int indiceDoVertice(int linha, int coluna) {
    return linha * largura + coluna;
}

void construirGrafo(void) {
    for (int linha = 0; linha < altura; linha++) {
        for (int coluna = 0; coluna < largura; coluna++) {
            if (mapa[linha][coluna] == '@') {
                continue;
            }
            int origem = indiceDoVertice(linha, coluna);
            for (int d = 0; d < 4; d++) {
                int novaLinha = linha + deltaLinha[d];
                int novaColuna = coluna + deltaColuna[d];
                if (novaLinha < 0 || novaLinha >= altura ||
                    novaColuna < 0 || novaColuna >= largura) {
                    continue;
                }
                if (mapa[novaLinha][novaColuna] == '@') {
                    continue;
                }
            }
        }
    }
}
```

```

        int destino = indiceDoVertice(novaLinha, novaColuna);
        adicionarAresta(origem, destino, letraDoMovimento[d]);
    }
}
}

void bfs(int verticeInicial, int verticeFinal, int totalDeVertices) {
    for (int i = 0; i < totalDeVertices; i++) {
        distancia[i] = -1;
    }
    int inicioDaFila = 0, fimDaFila = 0;
    fila[fimDaFila] = verticeInicial;
    fimDaFila++;
    distancia[verticeInicial] = 0;

    while (inicioDaFila < fimDaFila) {
        int verticeAtual = fila[inicioDaFila];
        inicioDaFila++;
        if (verticeAtual == verticeFinal) {
            break;
        }
        Aresta *aresta = listaDeAdjacencia[verticeAtual];
        while (aresta != NULL) {
            if (distancia[aresta->destino] == -1) {
                distancia[aresta->destino] = distancia[verticeAtual] + 1;
                veioDe[aresta->destino] = verticeAtual;
                movimentoParaChegar[aresta->destino] = aresta->movimento;
                fila[fimDaFila] = aresta->destino;
                fimDaFila++;
            }
            aresta = aresta->proxima;
        }
    }
}

int main(void) {
    scanf("%d %d", &altura, &largura);
    int linhaAluno, colunaAluno, linhaRU, colunaRU;

    for (int linha = 0; linha < altura; linha++) {
        for (int coluna = 0; coluna < largura; coluna++) {
            scanf(" %c", &mapa[linha][coluna]);
            if (mapa[linha][coluna] == 'A') {
                linhaAluno = linha;
                colunaAluno = coluna;
            }
            if (mapa[linha][coluna] == 'R') {
                linhaRU = linha;
                colunaRU = coluna;
            }
        }
    }

    construirGrafo();

    int verticeInicial = indiceDoVertice(linhaAluno, colunaAluno);
    int verticeFinal = indiceDoVertice(linhaRU, colunaRU);
    int totalDeVertices = altura * largura;

    bfs(verticeInicial, verticeFinal, totalDeVertices);

    if (distancia[verticeFinal] == -1) {
        printf("VOU MORRER DE FOME\n");
        return 0;
    }

    int totalDePassos = distancia[verticeFinal];
    char caminho[1000001];
    int verticeAtual = verticeFinal;
    int posicao = totalDePassos - 1;

    while (verticeAtual != verticeInicial) {
        caminho[posicao] = movimentoParaChegar[verticeAtual];
        posicao--;
    }
}

```

```
        verticeAtual = veioDe[verticeAtual];
    }
    caminho[totalDePassos] = '\\0';

    printf("AMEM\\n%s\\n%d\\n", caminho, totalDePassos);
    return 0;
}
```

Explicacao parte por parte

1. Includes e constante MAX_VERTICES

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_VERTICES 1000000
```

O **stdio.h** traz as funcoes de entrada e saida (scanf, printf). O **stdlib.h** traz o malloc, usado para alocar dinamicamente cada aresta do grafo. A constante MAX_VERTICES define o maior numero possivel de vertices: como o enunciado garante $N, M \leq 1000$, o grid tem no maximo $1000 \times 1000 = 1.000.000$ celulas, entao esse e o tamanho reservado para os vetores que guardam informacao por vertice.

2. A struct Aresta (representacao do grafo)

```
typedef struct Aresta {
    int destino;
    char movimento;
    struct Aresta *proxima;
} Aresta;

Aresta *listaDeAdjacencia[MAX_VERTICES];
```

Essa e a peca central do trabalho de grafos. Cada Aresta representa uma ligacao entre dois vertices: **destino** guarda para qual vertice a aresta aponta, **movimento** guarda a letra (C, B, E ou D) associada a esse deslocamento, e **proxima** aponta para a proxima aresta da mesma lista, formando uma lista ligada. O vetor **listaDeAdjacencia** e o grafo propriamente dito: a posicao i desse vetor guarda a lista ligada de todas as arestas que saem do vertice i . Essa forma de representar um grafo se chama **lista de adjacencia**, e e a alternativa mais comum a matriz de adjacencia quando o grafo e esparso (poucas arestas por vertice), como e o caso aqui (no maximo 4 arestas por vertice).

3. Vetores auxiliares e variaveis globais

```
char mapa[1000][1001];
int distancia[MAX_VERTICES];
int veioDe[MAX_VERTICES];
char movimentoParaChegar[MAX_VERTICES];
int fila[MAX_VERTICES];

int altura, largura;

int deltaLinha[4] = {-1, 1, 0, 0};
int deltaColuna[4] = {0, 0, -1, 1};
char letraDoMovimento[4] = {'C', 'B', 'E', 'D'};
```

mapa guarda o grid original lido da entrada, com os caracteres '.', '@', 'A' e 'R'. **distancia** guarda, para cada vertice, a menor distancia (em passos) ate ele a partir do vertice inicial, comecando todos em -1 (nao visitado). **veioDe** guarda de qual vertice veio o caminho minimo ate cada vertice, o que permite reconstruir o caminho no final. **movimentoParaChegar** guarda qual letra foi usada para chegar em cada vertice. **fila** e a fila usada pelo BFS. Os vetores **deltaLinha** e **deltaColuna** representam as 4 direcoes possiveis (cima, baixo, esquerda, direita), e **letraDoMovimento** guarda a letra correspondente a cada uma dessas direcoes, na mesma ordem.

4. A funcao adicionarAresta

```
void adicionarAresta(int origem, int destino, char movimento) {
    Aresta *nova = (Aresta *) malloc(sizeof(Aresta));
    nova->destino = destino;
    nova->movimento = movimento;
    nova->proxima = listaDeAdjacencia[origem];
    listaDeAdjacencia[origem] = nova;
}
```

Essa funcao cria uma aresta nova (com malloc) do vertice **origem** ate o vertice **destino**, com o movimento associado, e insere essa aresta no **inicio** da lista ligada de arestas de origem. Inserir no inicio e o jeito mais simples e mais rapido de adicionar um elemento numa lista ligada (custa tempo constante, nao precisa percorrer a lista inteira).

5. A funcao indiceDoVertice

```
int indiceDoVertice(int linha, int coluna) {
    return linha * largura + coluna;
}
```

O grafo trabalha com vertices identificados por um unico numero inteiro, mas o mapa e bidimensional (linha, coluna). Essa funcao faz a conversao entre os dois mundos: ela transforma uma posicao (linha, coluna) do grid num indice unico, contando as celulas linha por linha. Por exemplo, numa grade de largura 8, a celula da linha 2, coluna 3 vira o vertice numero $2 * 8 + 3 = 19$. Essa e a mesma logica usada para guardar uma matriz dentro de um vetor unidimensional.

6. A funcao construirGrafo

```
void construirGrafo(void) {
    for (int linha = 0; linha < altura; linha++) {
        for (int coluna = 0; coluna < largura; coluna++) {
            if (mapa[linha][coluna] == '@') {
                continue;
            }
            int origem = indiceDoVertice(linha, coluna);
            for (int d = 0; d < 4; d++) {
                int novaLinha = linha + deltaLinha[d];
                int novaColuna = coluna + deltaColuna[d];
                if (novaLinha < 0 || novaLinha >= altura ||
                    novaColuna < 0 || novaColuna >= largura) {
                    continue;
                }
                if (mapa[novaLinha][novaColuna] == '@') {
                    continue;
                }
                int destino = indiceDoVertice(novaLinha, novaColuna);
                adicionarAresta(origem, destino, letraDoMovimento[d]);
            }
        }
    }
}
```

Essa e a funcao que efetivamente **monta o grafo**, transformando o grid num conjunto de vertices e arestas. Ela percorre cada celula do mapa; se a celula for um obstaculo ('@'), ela e ignorada, pois obstaculos nao viram vertices. Para cada celula livre, a funcao calcula seu indice (origem) e tenta criar uma aresta para cada um dos 4 vizinhos possiveis (cima, baixo, esquerda, direita). Uma aresta so e criada se o vizinho estiver dentro dos limites do mapa e tambem nao for obstaculo. Ao final dessa funcao, o grafo inteiro ja esta pronto: todo vertice tem sua lista de arestas montada, e nenhuma busca foi feita ainda.

7. A funcao bfs

```
void bfs(int verticeInicial, int verticeFinal, int totalDeVertices) {
    for (int i = 0; i < totalDeVertices; i++) {
        distancia[i] = -1;
    }
    int inicioDaFila = 0, fimDaFila = 0;
    fila[fimDaFila] = verticeInicial;
    fimDaFila++;
    distancia[verticeInicial] = 0;

    while (inicioDaFila < fimDaFila) {
        int verticeAtual = fila[inicioDaFila];
        inicioDaFila++;
        if (verticeAtual == verticeFinal) {
            break;
        }
        Aresta *aresta = listaDeAdjacencia[verticeAtual];
        while (aresta != NULL) {
            if (distancia[aresta->destino] == -1) {
                distancia[aresta->destino] = distancia[verticeAtual] + 1;
                veioDe[aresta->destino] = verticeAtual;
                movimentoParaChegar[aresta->destino] = aresta->movimento;
                fila[fimDaFila] = aresta->destino;
                fimDaFila++;
            }
            aresta = aresta->proxima;
        }
    }
}
```

Essa e a **busca em largura** (BFS), o algoritmo de grafo que efetivamente resolve o problema. Primeiro, todas as distancias sao reiniciadas para -1 (ninguem foi visitado ainda). A fila comeca contendo apenas o vertice inicial, com distancia 0. Depois, o laco principal vai retirando um vertice da fila por vez (sempre o mais antigo, por isso e uma fila e nao uma pilha) e percorrendo sua **lista de adjacencia** (o ponteiro **aresta**, avancando por **aresta->proxima**). Para cada vizinho ainda nao visitado (distancia == -1), a funcao marca sua distancia como distancia do atual mais 1, guarda de onde ele veio e qual movimento foi usado, e coloca esse vizinho no fim da fila. Como o BFS sempre expande primeiro os vertices mais proximos da origem, a primeira vez que o vertice final e alcancado, garantidamente foi pelo caminho mais curto possivel. Assim que o vertice final sai da fila, o laco para, pois nao ha mais necessidade de continuar buscando.

8. Funcao main: leitura do mapa

```
int main(void) {
    scanf("%d %d", &altura, &largura);
    int linhaAluno, colunaAluno, linhaRU, colunaRU;

    for (int linha = 0; linha < altura; linha++) {
        for (int coluna = 0; coluna < largura; coluna++) {
            scanf(" %c", &mapa[linha][coluna]);
            if (mapa[linha][coluna] == 'A') {
                linhaAluno = linha;
                colunaAluno = coluna;
            }
            if (mapa[linha][coluna] == 'R') {
                linhaRU = linha;
                colunaRU = coluna;
            }
        }
    }
}
```

Essa parte lê a altura e a largura do mapa, e depois lê o mapa caractere por caractere. O espaço antes do `%c` no `scanf` ("`%c`") é importante: ele faz o `scanf` pular espaços em branco e quebras de linha antes de ler o próximo caractere, evitando que o programa leia acidentalmente um caractere de nova linha no lugar de '.', '@', 'A' ou 'R'. Enquanto lê, o código também guarda a posição onde encontrou 'A' (o aluno, ponto de partida) e 'R' (o RU, destino).

9. Função main: construir o grafo e rodar o BFS

```
construirGrafo();

int verticeInicial = indiceDoVertice(linhaAluno, colunaAluno);
int verticeFinal = indiceDoVertice(linhaRU, colunaRU);
int totalDeVertices = altura * largura;

bfs(verticeInicial, verticeFinal, totalDeVertices);
```

Aqui fica clara a separação entre as duas etapas do trabalho: primeiro **construirGrafo()** monta toda a estrutura do grafo (vértices e arestas), e só depois **bfs()** é chamada para efetivamente buscar o caminho mais curto dentro desse grafo já pronto. As posições (linha, coluna) do aluno e do RU são convertidas em índices de vértice usando a função `indiceDoVertice`, para que o BFS possa trabalhar apenas com números de vértice, sem precisar saber nada sobre linhas e colunas.

10. Função main: resultado e reconstrução do caminho

```
if (distancia[verticeFinal] == -1) {
    printf("VOU MORRER DE FOME\n");
    return 0;
}

int totalDePassos = distancia[verticeFinal];
char caminho[1000001];
int verticeAtual = verticeFinal;
int posicao = totalDePassos - 1;

while (verticeAtual != verticeInicial) {
    caminho[posicao] = movimentoParaChegar[verticeAtual];
    posicao--;
    verticeAtual = veioDe[verticeAtual];
}
caminho[totalDePassos] = '\0';

printf("AMEM\n%s\n%d\n", caminho, totalDePassos);
return 0;
}
```

Se a distância do vértice final continuar -1 após o BFS, isso significa que o RU nunca foi alcançado, então não existe caminho possível, e o programa imprime "VOU MORRER DE FOME". Caso contrário, o caminho precisa ser reconstruído. Isso é feito de trás para frente: começando no vértice final, o código usa o vetor **veioDe** para "andar para trás" até voltar ao vértice inicial, guardando cada movimento na posição correta do vetor **caminho** (por isso a variável **posicao** começa em `totalDePassos - 1` e vai diminuindo). Ao final, o vetor **caminho** contém a sequência de movimentos na ordem correta (do início para o fim), e o programa imprime "AMEM", a sequência de movimentos e o total de passos.